

DEEPL - TP 6 - Regularization and Experimental Protocol

Nicolas Baskiotis - Stéphane Rivaud - Benjamin Piwowarski - Laure Soulier

2025-2026

In this practical session, we will study the mechanisms that allow a model to generalize *better*.

- L1 and L2 regularization
- Dropout
- BatchNorm
- LayerNorm

For Dropout and BatchNorm, in both cases normalization behaves differently depending on whether the model is in inference or training mode. In **PyTorch**, to set the mode, you must call the module's `eval()` and `train()` methods.

This practical session is also an opportunity to introduce two very useful modules for experimentation : `lightning`, which greatly factors experimental code, and `optuna`, which allows the optimization of hyperparameter search.

1 Data and Model Preparation

You will use the MNIST dataset.

Important

In this session, you will use only 5% of the training data (otherwise regularization techniques will have no effect).

```
from datamaestro import prepare_dataset
ds = prepare_dataset("com.lecun.mnist")

# Do not forget to subsample!
train_img, train_labels = ds.train.images.data(), ds.train.labels.data()
test_img, test_labels = ds.test.images.data(), ds.test.labels.data()
```

You will use a network composed of 3 linear layers with 100 outputs, followed by a linear layer for classification (10 classes – digits from 0 to 9). You will use a cross-entropy loss, batch size of 300, and 1000 iterations (*epochs*).

Question 1

To observe the effects of regularization, you will need to record with `torch.utils.tensorboard` :

- The losses (train, validation, test).
- The weights of each linear layer (histogram).
- The gradient at the input of each linear layer; to record the gradient with respect to a variable, use the `store_grad` function or the tensor method `retain_grad`.
- The entropy of the output (histogram) – quickly compute the entropy of a random model as comparison.

Warning : Avoid logging histograms at every iteration; make sure to record at most around twenty during training to save storage space and computation time.

2 Model Regularization

L1 and L2 Regularization

L1 or L2 regularization corresponds to placing a prior on the parameters. Using a maximum a posteriori criterion — with a Laplace prior (for L1) or a Gaussian prior (for L2). Indeed, we want to maximize

$$p(\theta|\text{data}) = \frac{p(\text{data}|\theta)p(\theta)}{p(\text{data})}$$

which amounts to minimizing

$$\log p(\theta|\text{data}) = \text{constant} - \log p(\text{data}|\theta) - \log p(\theta)$$

For a Laplace prior $p(\theta)$, we have :

$$\log p(\theta) = \text{constant} + \underbrace{\frac{1}{b}}_{\lambda_1} ||\theta||_1$$

For a Gaussian prior $p(\theta)$, we have :

$$\log p(\theta) = \text{constant} + \underbrace{\frac{1}{2\sigma^2}}_{\lambda_2} ||\theta||_2^2$$

Thus, there is a hyperparameter to tune for L1/L2 regularization. In practice, the larger it is, the less the parameters deviate from 0. Observe the effects.

Dropout

The idea behind Dropout is close to model averaging : by training k independent models, we reduce model variance. Training k models is computationally expensive, and Dropout aims at reducing memory and computation cost. The idea is to set each output of a Dropout layer to zero with probability p (a hyperparameter).

Important

During inference, this probability p must be ignored (and values adjusted since all inputs are present, by multiplying by $(1-p)^{-1}$). To do this, you must specify whether the model is training (with `model.train()`) or performing inference (with `model.eval()`).

BatchNorm

It is known that centered and normalized data allow faster and more stable training; while we can ensure that input data are centered and normalized, it is more difficult for internal layers of a neural network. BatchNorm consists in adding a layer whose purpose is to center/normalize data using a moving average/variance (during inference) and batch statistics (during training).

Use the `BatchNorm1d` class.

Important

Again, there is a distinct training and inference mode : in the latter, the moving average/variance is no longer updated.

LayerNorm

LayerNorm is an alternative to BatchNorm, mainly used when there is little or no data to estimate variance and mean (such as in reinforcement learning). It normalizes data per individual (and not per feature as in BatchNorm).

Use the `LayerNorm` class.

Question 2

Modify your model to use one of the regularization techniques proposed above. Some can be combined (e.g., L1/L2 with Dropout, BatchNorm, or LayerNorm).

3 Data Augmentation (optional)

Another technique to prevent overfitting is to artificially create more data. You may use the functions available in torchvision.

Question 3

To do so, you may use transformations that *do not change the expected class* :

- Adding Gaussian noise
- In the case of images, non-destructive geometric transformations (rotations, zoom, etc.)

4 Lightning

Lightning makes it possible to factorize the experimental part of your code and automate many redundant operations. In particular, it allows :

- simplifying the training loop, checkpointing, logging
- being hardware agnostic (GPU, multi-GPU, CPU)
- switching between training/evaluation/test modes smoothly

Two classes are generally needed : a class implementing your model's behavior, inherited from `LightningModule`, and a class for data loading inherited from `LightningDataModule`.

A typical model code is shown in `lightningexample.py`. You find the usual `forward` method to compute the model output. The `configure_optimizers` method specifies the optimizer used. You only need to implement `training_step`, `validation_step`, and `test_step`, which perform one training, validation, and test iteration : at minimum, apply the model to the batch and compute the loss. You will notice that there is no explicit call to backward or the optimizer, nor to transfers between CPU/GPU : everything is encapsulated. You only need to create a `Trainer` and call its `fit` method on the model and data to train : all calls will be made automatically.

The `LitMnistData` class gives an example of dataset management : you need a `prepare_dataset` method and a `setup` method to set up the data (download, preprocess), and the `train_dataloader`, `val_dataloader`, and `test_dataloader` methods which return training, validation, and test data respectively.

Many *hooks* exist to make module use flexible, such as `training_epoch_end`, which allows performing custom computations at the end of an epoch (e.g., logging histograms, images, or other quantities). Read the documentation for more information.

5 Hyperparameter Search

You will attempt to search for the best hyperparameters using the remaining 95% of the training set as validation. You may try to combine different forms of regularization. Many libraries exist (e.g. `nevergrad`, still in development), but we recommend *optuna*, which easily supports dependencies between hyperparameters.

Question 4

Implement a hyperparameter search using the *optuna* library for hyperparameter optimization.